

Document number: P0025R1

Date: 2015-10-29

Revises: P0025R0

Project: Programming Language C++, Library Working Group

Reply to: Martin Moene <martin.moene (at) gmail.com>, Niels Dekker <n.dekker (at) xs4all.nl>

An algorithm to "clamp" a value between a pair of boundary values (revision 2)

Changes since P0025R0

- The requirement for `lo` to be no greater than `hi` has been added per guidance from SG6 (Numerics) and LEWG.
- The example using the predicate form has been replaced.
- The name *limit* has been removed in favor of P0105, Rounding and Overflow in C++.
- A brief discussion of `middle()` and `median()` has been added.

Changes since N4536

- Function `clamp_range()` is considered superfluous in view of the Ranges proposal and has been dropped from this proposal.
- The declaration style of `clamp()` has been made consistent with the one of `min()` and `max()`.

Contents

[Introduction](#)

[Motivation](#)

[Impact on the standard](#)

[Comparison to clamp of Boost.Algorithm](#)

[Design decisions](#)

[Proposed wording](#)

[Possible implementation](#)

[Acknowledgments](#)

[References](#)

Introduction

The algorithm proposed here "clamps" a value between a pair of boundary values. The idea and interfaces are inspired by `clamp` in the `Boost.Algorithm` library authored by Marshall Clow.

Motivation

It is a common programming task to constrain a value to fall within certain limits. This can be expressed in numerous ways, but it would be good if such an operation can be easily recognized and doesn't appear in many guises just because it can.

So, we'd like to have a concise way to obtain a value that is forced to fall within a range we request, much like we can limit a value to a defined minimum or maximum. For example:

```
auto clamped_value = clamp( value, min_value, max_value );
```

Without a standardized way, people may (need to) define their own version of "clamp" or resort to a less clear solution such as¹:

```
auto clamped_value = std::min( std::max( value, min_value ), max_value );
```

In addition to the boundary values, one can provide a predicate that evaluates if a value is within the boundary.

```
// Clamp according to default, alphabetic order: yields "10"
auto clamped_alphabetic = clamp("10"s, "0"s, "9"s);

// Clamp according to predicated, numeric order: yields "9"
auto clamped_numeric = clamp("10"s, "0"s, "9"s,
    [](const auto& lhs, const auto& rhs) { return stoi(lhs) < stoi(rhs); } );
```

Function `clamp()` already exists in C++ libraries such as Boost [1] and Microsoft AMP [2]. The Qt Project provides `qBound()` [3], and the Python library `scipy/numpy` provides `clip()` [4] for the same purpose.

Impact on the standard

The clamp algorithms can be implemented as a pure library extension. We suggest to add them to sub-clause 25.4 Sorting and related operations of the Algorithms library.

Comparison to clamp of Boost.Algorithm

Like `std::min()` and `std::max()`, `clamp()` requires its arguments to be of the same type, whereas Boost's clamp accepts arguments of different type.

Design decisions

We chose the name *clamp* as it is expressive and is already being used in other libraries [2](#).

`clamp()` can be regarded as a sibling of `std::min()` and `std::max()`. This makes it desirable to follow their interface using `constexpr`, passing parameters by const reference and returning the result by const reference. Passing values by `const &` is desired for types that have a possibly expensive copy constructor such as `cpp_int` of Boost.Multiprecision [\[5\]](#) and `std::seminumeric::integer` from the Proposal for Unbounded-Precision Integer Types [\[6\]](#).

It has been noted that a new function like `middle(a, b, c)` or `median(a, b, c)` could play the role of `clamp(a, b, c)`, removing the requirement on parameter order [\[7\]](#). This would increase the number of required comparisons [3](#), as would only removing the requirement on the order of the boundary values of `clamp()` [4](#). It is likely that `clamp()` will be avoided in many use cases if it is more expensive to use than the combination of `min()` and `max()`, so we do not proceed along this path here.

Proposed wording

25.4.X Bounded value

[alg.clamp]

```
template<class T>
constexpr const T& clamp( const T& v, const T& lo, const T& hi );

template<class T, class Compare>
constexpr const T& clamp( const T& v, const T& lo, const T& hi, Compare comp );
```

1 *Requires*: The value of `lo` shall be no greater than `hi`. For the first form, type `T` shall be `LessThanComparable` (Table 18).

2 *Returns*: The larger value of `v` and `lo` if `v` is smaller than `hi`, otherwise the smaller value of `v` and `hi`.

3 *Remarks*: Returns `v` when it is equivalent to `lo`, `hi`, or both.

4 *Complexity*: At most two comparisons.

Possible implementation

Clamp a value:

```
template<class T>
constexpr const T& clamp( const T& v, const T& lo, const T& hi )
{
    return clamp( v, lo, hi, less<T>() );
}
```

Clamp a value per predicate:

```
template<class T, class Compare>
constexpr const T& clamp( const T& val, const T& lo, const T& hi, Compare comp )
{
    return assert( !comp(hi, lo) ),
           comp(val, lo) ? lo : comp(hi, val) ? hi : val;
}
```

Acknowledgments

Thanks to Marshall Clow for Boost.Algorithm's clamp which inspired this proposal, to the BSI C++ panel for their feedback and to Daniel Krüglér, Jonathan Wakely and Lawrence Crowl for their help with the proposing process and to Walter Brown for his offer to shepherd the paper through LWG.

References

[1] Marshall Clow. [clamp in the Boost Algorithm Library](#).

Note: the Boost documentation shows `clamp()` using pass by value, whereas the actual code in [boost/algorithm/clamp.hpp](#) uses

const &. See [ticket 10081](#).

[2] Microsoft. [C++ Accelerated Massive Parallelism library \(AMP\)](#).

[3] Qt Project. [Documentation on qBound](#).

[4] Scipy.org. [Documentation on numpy.clip](#).

[5] John Maddock. [Boost.Multiprecision](#).

[6] Pete Becker. [Proposal for Unbounded-Precision Integer Types \(N4038\)](#).

[7] Walter Brown. Post Kona review observation. Personal communication, 25 October 2015.

1. Or even:

```
auto clamped_value = value;
if ( value < min_value ) clamped_value = min_value;
else if ( value > max_value ) clamped_value = max_value;
```

[←](#)

2. As suggested by Jonathan Wakely on mailing list `accu-general` on 18 February 2014.[↪](#)

3. `median()` expressed via `min()` and `max()`:

```
template<class T>
constexpr const T& median( const T& a, const T& b, const T& c )
{
    return max( min(a, b), min( max(a, b), c ) );
}
```

[←](#)

4. `clamp()` with free boundary order expressed via `min()` and `max()`:

```
template<class T>
constexpr const T& clamp( const T& v, const T& bound1, const T& bound2 )
{
    return min( max( v, min(bound1, bound2) ), max(bound1, bound2) );
}
```

[←](#)